



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

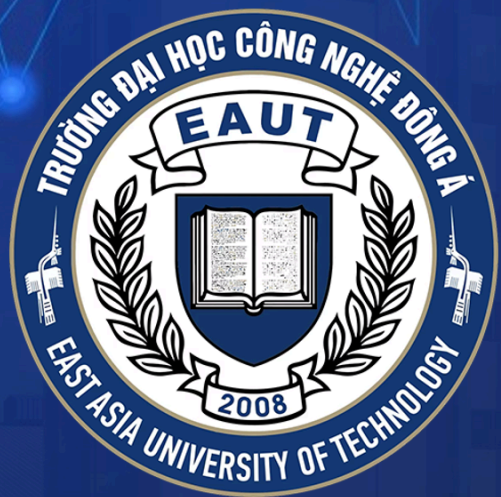
KHOA CÔNG NGHỆ THÔNG TIN

XỬ LÝ ẢNH & THỊ GIÁC MÁY TÍNH

CHƯƠNG 3: NÉN ẢNH

Giảng viên: Nguyễn Phồn Lữ





NỘI DUNG

- Giới thiệu về nén dữ liệu
- Nén không tổn thất (Lossless)
- Nén có tổn thất (Lossy)



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

GIỚI THIỆU VỀ NÉN DỮ LIỆU



GIỚI THIỆU

- **Khái niệm:** Nén ảnh là quá trình giảm dung lượng (số bit) cần thiết để biểu diễn một bức ảnh, sao cho ảnh sau khi nén vẫn đảm bảo chất lượng chấp nhận được theo mục đích sử dụng.
- **Ví dụ:** Một bức ảnh màu 10 megapixel. Nếu lưu dạng thô, mỗi pixel gồm 3 kênh (R, G, B), mỗi kênh 8 bit.
 - Tổng dung lượng: $10M \times 3 \times 8 = 240.000.000 \text{ bit} \approx 30 \text{ MB}$.
 - Nén ảnh giúp giảm kích thước này xuống còn vài MB hoặc vài trăm KB mà mắt thường khó phân biệt được sự khác biệt.
- **Tại sao cần nén dữ liệu?**
 - **Tiết kiệm bộ nhớ lưu trữ:** Giảm dung lượng chiếm dụng trên ổ cứng, thẻ nhớ, hoặc cloud.
 - **Tăng tốc truyền tải:** Giúp việc gửi ảnh qua mạng, tải trang web, hoặc gọi video diễn ra nhanh chóng hơn.
 - **Giảm băng thông:** Yếu tố cực kỳ quan trọng trong các ứng dụng streaming video, hội nghị truyền hình thời gian thực.

DỮ LIỆU, THÔNG TIN, TỶ LỆ NÉN

- **Dữ liệu (Data):** Phương tiện mang thông tin.
- **Thông tin (Information):** Nội dung thực sự cần truyền tải.
- **Nén dữ liệu:** Giảm số bit dùng để biểu diễn dữ liệu nhưng vẫn giữ được thông tin quan trọng.
- **Dư thừa dữ liệu (Redundancy - R):** Đo phần dữ liệu không cần thiết. Nếu C là tỷ lệ nén $\Rightarrow R = 1 - \frac{1}{C}$. R càng lớn, dữ liệu càng có nhiều phần có thể loại bỏ hoặc mã hóa hiệu quả.
- **Tỷ lệ nén (Compression Ratio - C):** $C = \frac{b}{b'}$, với:
 - b là số bit gốc,
 - b' là số bit sau nén.
- *Ví dụ:* $C = 10 : 1 \Rightarrow R = 0.9$ (90% dữ liệu là dư thừa).

CÁC LOẠI DƯ THỪA DỮ LIỆU TRONG ẢNH SỐ

- **Dư thừa mã hóa (Coding Redundancy):** Sử dụng nhiều bit hơn mức cần thiết để biểu diễn các mức xám do phân bố xác suất không đều.
- **Dư thừa không gian (Spatial Redundancy):** Các pixel lân cận có giá trị rất giống nhau hoặc phụ thuộc lẫn nhau mạnh. Thường được khai thác bởi RLE, DCT, hoặc các bộ lọc không gian/tần số.
- **Thông tin không liên quan (Irrelevant Information):** Thông tin bị hệ thống thị giác con người (HVS) bỏ qua hoặc không cần thiết cho ứng dụng cụ thể. Thường bị loại bỏ trong nén mất mát (lossy).

ĐO LƯỜNG THÔNG TIN ẢNH (ENTROPY)

- **Entropy:** Lượng tin trung bình trên mỗi pixel, đo mức độ không chắc chắn của phân bố mức xám.
- **Lượng tin của sự kiện E:** $I(E) = \log_2 \frac{1}{P(E)}$ (bit). Sự kiện càng ít xảy ra thì lượng tin càng cao.
- **Entropy của nguồn:** $H = - \sum_{k=0}^{L-1} p(r_k) \log_2 p(r_k)$ (với L là số mức cường độ).
- **Ý nghĩa:** Ảnh càng đồng đều, ít quy luật thì entropy càng cao. Ảnh có nhiều vùng lặp lại thì entropy thấp.
- **Định lý Shannon 1:** Không thể mã hóa nguồn với số bit trung bình nhỏ hơn entropy H .

THỰC HÀNH TÍNH ENTROPY CỦA ẢNH.

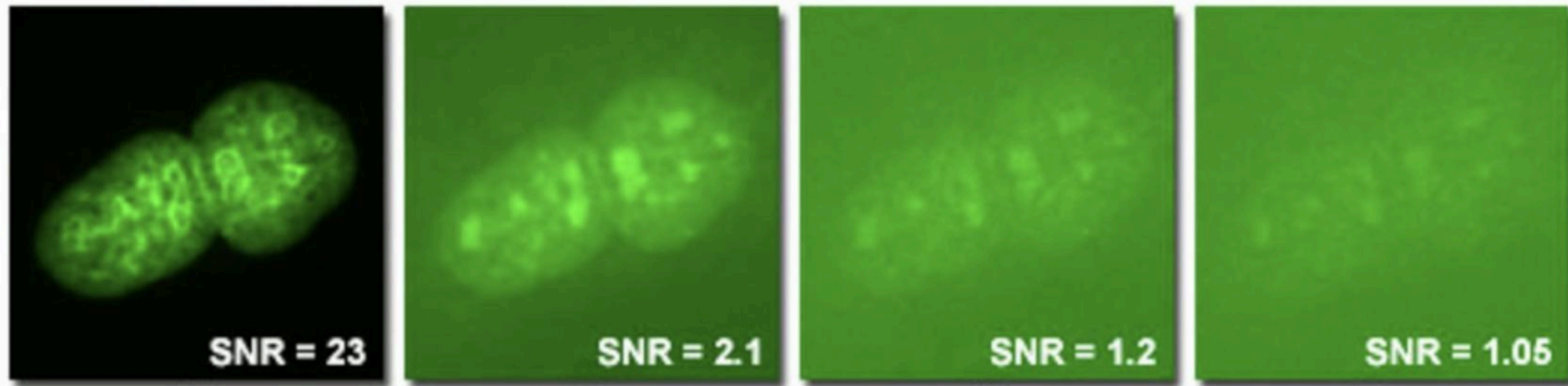
```
import math

def calculate_entropy(probabilities):
    entropy = 0
    for p in probabilities:
        if p > 0:
            entropy -= p * math.log2(p)
    return entropy

# Ví dụ: Ảnh có 3 mức xám với xác suất xuất hiện lần lượt là 0.5, 0.3, 0.2
probs = [0.5, 0.3, 0.2]
print(f"Entropy của ảnh: {calculate_entropy(probs):.4f} bits/pixel")
```

TIÊU CHUẨN ĐÁNH GIÁ CHẤT LƯỢNG (FIDELITY CRITERIA)

- **Đánh giá khách quan:** Dựa trên công thức đo lường.
 - **RMSE (Root Mean Square Error):** Đo mức sai khác trung bình.
$$e_{rms} = \sqrt{\frac{1}{MN} \sum \sum (f(x, y) - \hat{f}(x, y))^2}.$$
 RMSE càng nhỏ, ảnh càng gần gốc.
 - **SNR (Signal-to-Noise Ratio):** Tỷ lệ tín hiệu trên nhiễu. SNR càng cao, chất lượng ảnh càng tốt.
- **Đánh giá chủ quan:** Đánh giá bằng mắt thường (Thang điểm 1-6: Excellent đến Unusable).



THỰC HÀNH TÍNH RMSE VÀ SNR.

```
import numpy as np

def calculate_rmse(original, compressed):
    return np.sqrt(np.mean((original - compressed) ** 2))

def calculate_snr(original, compressed):
    signal_power = np.mean(original ** 2)
    noise_power = np.mean((original - compressed) ** 2)
    return 10 * np.log10(signal_power / noise_power)

# Ví dụ
original = np.array([[100, 150], [200, 250]])
compressed = np.array([[105, 145], [195, 255]])
print(f"RMSE: {calculate_rmse(original, compressed):.4f}")
print(f"SNR: {calculate_snr(original, compressed):.2f} dB")
```

CÁC LOẠI NÉN ẢNH

- **Nén không tổn hao (Lossless):**

- *Nguyên lý:* Tìm và loại bỏ sự dư thừa mà không làm mất thông tin.
- *Đặc điểm:* Ảnh khôi phục giống hệt ảnh gốc. Tỷ lệ nén thấp (thường 2:1 đến 4:1).
- *Ứng dụng:* Ảnh y tế, văn bản, ảnh đồ họa (logo).

- **Nén có tổn hao (Lossy):**

- *Nguyên lý:* Loại bỏ những thông tin mà mắt người khó nhận ra.
- *Đặc điểm:* Ảnh khôi phục gần giống, mất một số chi tiết nhỏ. Tỷ lệ nén cao (10:1, 20:1 hoặc hơn).
- *Ứng dụng:* Ảnh chụp, video, web (JPEG).

MÔ HÌNH HỆ THỐNG NÉN ẢNH TỔNG QUÁT

- Hệ thống nén ảnh thường được chia thành hai khối chính: bộ mã hóa và bộ giải mã. Bộ mã hóa biến ảnh gốc thành dạng dữ liệu ngắn gọn hơn để lưu trữ hoặc truyền đi, còn bộ giải mã khôi phục lại ảnh từ dữ liệu đã nén.
- **Bộ mã hóa (Encoder):**
 - **Mapper:** Giảm dư thừa không gian/thời gian (biến đổi ảnh sang dạng thuận lợi hơn).
 - **Quantizer:** Giảm thông tin không liên quan (chỉ có trong nén lossy). Làm tròn/gộp giá trị để giảm số mức biểu diễn.
 - **Symbol Coder:** Giảm dư thừa mã hóa (sử dụng mã độ dài biến đổi, mã loạt dài...).
- **Bộ giải mã (Decoder):** Thực hiện quá trình ngược lại.
 - **Symbol Decoder:** Giải mã ký hiệu.
 - **Inverse Quantizer:** Giải lượng tử hóa (chỉ có trong lossy).
 - **Inverse Mapper:** Biến đổi ngược để khôi phục ảnh.



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

NÉN KHÔNG TỔN THẤT (LOSSLESS)

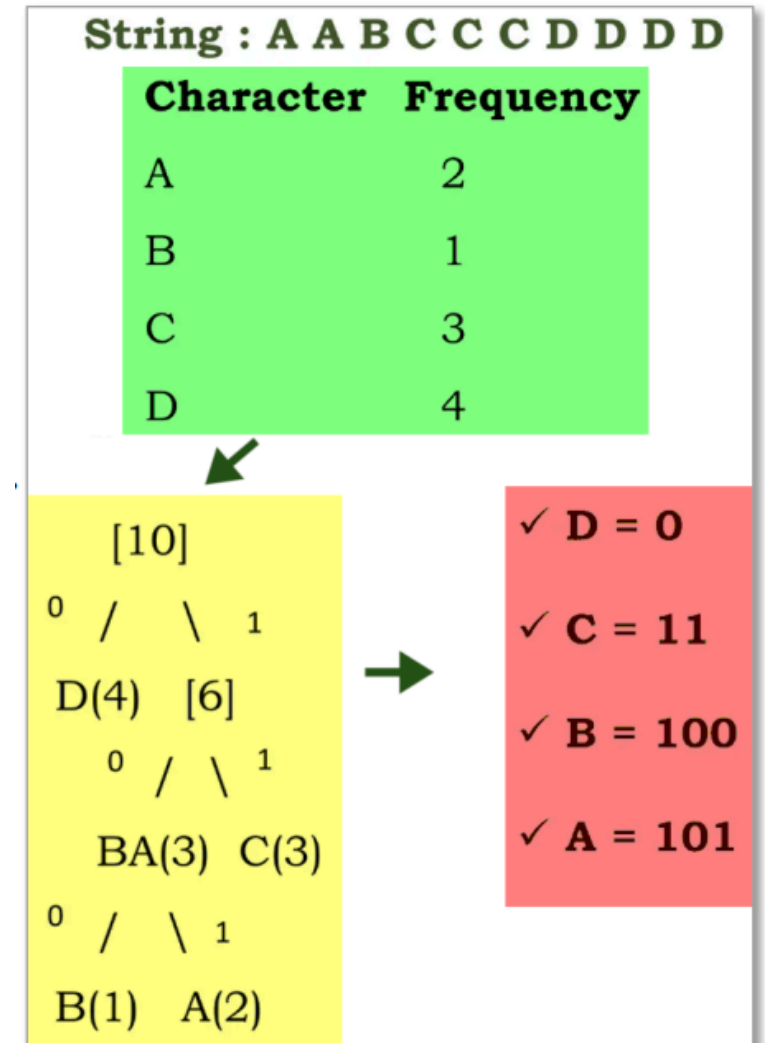


ĐỊNH NGHĨA

- Các thuật toán và kỹ thuật nén ảnh bảo toàn toàn vẹn dữ liệu gốc.
- Đảm bảo ảnh giải nén giống hệt 100% so với ảnh trước khi nén.

MÃ HUFFMAN

- **Khái niệm:** Phương pháp mã hóa độ dài thay đổi. Ký tự xuất hiện càng nhiều thì mã càng ngắn, ký tự hiếm mã càng dài. Giúp giảm số bit trung bình cần dùng.
- **Quy trình xây dựng cây Huffman:**
 1. Sắp xếp các ký tự theo xác suất (tần suất).
 2. Gộp 2 xác suất nhỏ nhất thành một nút mới có giá trị bằng tổng 2 nút con.
 3. Lặp lại cho đến khi còn lại 1 cây duy nhất.
 4. Gán bit cho các nhánh (ví dụ: trái = 0, phải = 1).
- **Ứng dụng:** Mã hóa các pixel theo giá trị mức xám (0-255).



VÍ DỤ MÃ HUFFMAN (1)

- **Chuỗi cần mã hoá:** "Cộng hoà xã hội chủ nghĩa"
- **Trước mã hoá:** Chuỗi ký tự ASCII tiêu chuẩn (mỗi ký tự 8 bit).
- **Sau mã hoá:** Chuỗi bit được rút gọn đáng kể nhờ các ký tự xuất hiện nhiều (như dấu cách, chữ 'a', 'o', 'h') được gán mã ngắn.

Chuỗi cần mã hoá: “Cộng hoà xã hội chủ nghĩa”, trước mã hoá:

01000011111000011011101110011001011011100110011100100000011
01000011011111100001110100000001000000111100011000011101000
11001000000110100011100001101110111001100101101001001000000
11000110110100011100001101110111010011100100000011011100110
011101101000110001001010100101100001

Sau mã hoá:

10101110111001111010010100101110110110100010100110110000011
001100100100111001111001110111100

- *Lưu ý:* Cây Huffman hoặc bảng mã cần được lưu kèm hoặc gửi kèm ở phần đầu file nén để bộ giải mã có thể hiểu được.

THỰC HÀNH XÂY DỰNG CÂY HUFFMAN VÀ MÃ HÓA CHUỖI.

```
import heapq

class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None
    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(text):
    freq = {}
    for char in text:
        freq[char] = freq.get(char, 0) + 1
    heap = [Node(char, f) for char, f in freq.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = Node(None, left.freq + right.freq)
        merged.left = left
        merged.right = right
        heapq.heappush(heap, merged)
    return heap[0]

def generate_codes(node, prefix="", code_map={}):
    if node is not None:
        if node.char is not None:
            code_map[node.char] = prefix
            generate_codes(node.left, prefix + "0", code_map)
            generate_codes(node.right, prefix + "1", code_map)
    return code_map

text = "aabbcc"
tree = build_huffman_tree(text)
codes = generate_codes(tree)
print("Mã Huffman:", codes)
```

GIẢI MÃ HUFFMAN

- **Nguyên lý:** Dựa vào cây Huffman đã được xây dựng để dịch ngược chuỗi bit về ký tự.
- **Quy trình:**
 1. Đọc lần lượt từng bit từ luồng dữ liệu nén.
 2. Bắt đầu từ nút gốc (root) của cây Huffman.
 3. Gặp bit 0 rẽ trái, gặp bit 1 rẽ phải.
 4. Khi đến nút lá (leaf), ta được một ký tự gốc. Xuất ký tự này ra.
 5. Quay lại nút gốc và tiếp tục đọc bit tiếp theo cho đến hết luồng dữ liệu.

MÃ GOLOMB & GOLOMB-RICE

- **Mã Golomb:**

- Ý tưởng: Mã hóa số nguyên N bằng cách chia cho M .
- Thương (Quotient) mã hóa phần đầu, Số dư (Remainder) mã hóa phần sau.
- Hiệu quả khi các số nhỏ xuất hiện nhiều hơn số lớn (phân bố mũ).

- **Mã Golomb-Rice:**

- Trường hợp đặc biệt khi $M = 2^k$.
- Phép chia trở thành phép dịch bit (rất nhanh).
- Thương = dịch phải k bit. Số dư = k bit thấp hơn.

$$N = qM + r$$

$$\text{Golomb Code} = \text{Unary}(q) + \text{Code}(r)$$

Golomb-Rice (GR) codes

Integer	Code			
	$k = 0$	$k = 1$	$k = 2$	$k = 3$
0	0•	0•0	0•00	0•000
1	10•	0•1	0•01	0•001
2	110•	10•0	0•10	0•010
3	1110•	10•1	0•11	0•011
4	11110•	110•0	10•00	0•100
5	111110•	110•1	10•01	0•101
6	1111110•	1110•0	10•10	0•110
7	11111110•	1110•1	10•11	0•111
8	111111110•	11110•0	110•00	10•000

GIẢI MÃ GOLOMB & GOLOMB-RICE

- **Nguyên lý:** Tách lại phần Thương và phần Dư để tính ra số nguyên ban đầu N .
- **Quy trình (với Golomb-Rice):**
 1. Đọc các bit cho đến khi gặp bit 1 đầu tiên. Số lượng bit 0 đếm được chính là Thương (q).
 2. Đọc tiếp k bit tiếp theo để lấy Số dư (r).
 3. Khôi phục giá trị gốc: $N = q \times 2^k + r$.

THỰC HÀNH MÃ HÓA VÀ GIẢI MÃ GOLOMB-RICE.

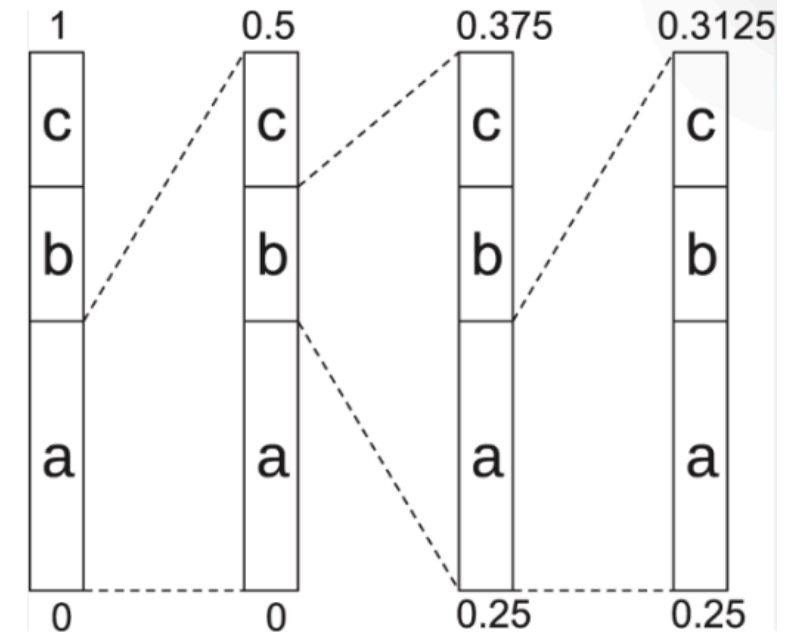
```
def golomb_rice_encode(n, k):
    M = 2 ** k
    q = n // M
    r = n % M
    quotient_bits = '0' * q + '1'
    remainder_bits = f"{r:0{k}b}"
    return quotient_bits + remainder_bits

def golomb_rice_decode(bits, k):
    q = 0
    for bit in bits:
        if bit == '0': q += 1
        else: break
    r_bits = bits[q+1 : q+1+k]
    r = int(r_bits, 2)
    return q * (2 ** k) + r

n = 13
k = 2
encoded = golomb_rice_encode(n, k)
print(f"Mã hóa {n} với k={k}: {encoded}")
print(f"Giải mã: {golomb_rice_decode(encoded, k)}")
```

MÃ SỐ HỌC (ARITHMETIC CODING)

- **Khác biệt với Huffman:** Không ánh xạ 1-1 giữa ký tự và mã. Một chuỗi ký tự dài được ánh xạ vào một khoảng số thực duy nhất trong $[0, 1)$.
- **Ý tưởng:** Mỗi ký tự làm thu hẹp dần một khoảng. Sau khi xử lý hết chuỗi, chỉ cần chọn một số nằm trong khoảng cuối cùng.
- **Ưu điểm:** Vượt qua giới hạn 1 bit/ký tự của Huffman, tiệm cận giới hạn Entropy của Shannon.
- **Mở rộng:** Mô hình xác suất thích nghi theo ngữ cảnh (Adaptive Context-Dependent), xác suất thay đổi theo ngữ cảnh trước đó giúp tăng hiệu quả nén.



VÍ DỤ MÃ SỐ HỌC

- Giả sử bảng xác suất: $A = 0.5$, $B = 0.3$, $C = 0.2$.

- Các khoảng tương ứng:

- $A: [0.0, 0.5)$
- $B: [0.5, 0.8)$
- $C: [0.8, 1.0)$

- Mã hóa chuỗi "AB":

- Ký tự 'A' (khoảng $[0.0, 0.5)$): Thu hẹp khoảng hiện tại thành $[0.0, 0.5)$.
- Ký tự 'B' (chiếm 30% khoảng): Chia khoảng $[0.0, 0.5)$ thành 3 phần. 'B' nằm ở khoảng $[0.5 \times 0.5, 0.8 \times 0.5) = [0.25, 0.4)$.

- Kết quả:** Bất kỳ số nào trong khoảng $[0.25, 0.4)$ (ví dụ: 0.3) đều đại diện cho chuỗi "AB".

Giả sử

- Bảng chữ cái gồm 3 ký tự: A, B, C.
- Xác suất xuất hiện:

$$P(A) = 0.5, P(B) = 0.3, P(C) = 0.2.$$

- Khoảng con ban đầu trên đoạn $[0, 1)$:

- A: $[0, 0.5)$
- B: $[0.5, 0.8)$
- C: $[0.8, 1.0)$

Mã hóa chuỗi "BAC"

Bước 1: Khoảng ban đầu $[0, 1)$

Ký tự đầu tiên là B → khoảng mới = $[0.5, 0.8)$

Độ rộng hiện tại $range = 0.8 - 0.5 = 0.3$.

Bước 2: Ký tự A

- Trong khoảng con của A: $[0, 0.5)$ so với khoảng hiện tại.
- Tính lại:

$$low = 0.5 + 0.3 \times 0 = 0.5$$

$$high = 0.5 + 0.3 \times 0.5 = 0.65$$

→ Khoảng mới: $[0.5, 0.65)$

$$range = 0.15.$$

Bước 3: Ký tự C

- Trong khoảng con của C: $[0.8, 1.0)$
- Tính:

$$low = 0.5 + 0.15 \times 0.8 = 0.5 + 0.12 = 0.62$$

$$high = 0.5 + 0.15 \times 1.0 = 0.65$$

→ Khoảng cuối cùng: $[0.62, 0.65)$

Chọn một số bất kỳ trong khoảng này, ví dụ 0.63.

Đó là giá trị mã hóa cho chuỗi "BAC".

Giải mã từ giá trị 0,63

- Bước 1: 0,63 nằm trong khoảng nào của $[0, 1)$?

$[0.5, 0.8)$ → ký tự B.

Cập nhật: $low = 0.5$, $range = 0.3$.

Tính $(0.63 - 0.5) / 0.3 = 0.13 / 0.3 \approx 0.4333$.

- Bước 2: 0,4333 nằm trong $[0, 0.5)$ → ký tự A.

Cập nhật: $low = 0.5$, $range = 0.15$.

Tính $(0.63 - 0.5) / 0.15 = 0.13 / 0.15 \approx 0.8667$.

- Bước 3: 0,8667 nằm trong $[0.8, 1.0)$ → ký tự C.

Giải mã xong, được chuỗi "BAC".

GIẢI MÃ SỐ HỌC (ARITHMETIC DECODING)

- **Nguyên lý:** Xác định xem con số thực nằm trong khoảng con của ký tự nào để suy ra ký tự đó.
- **Quy trình:**
 1. Khởi tạo khoảng hiện tại là $[0, 1)$.
 2. Dựa vào bảng xác suất, chia khoảng hiện tại thành các khoảng con.
 3. Kiểm tra xem con số giải mã nằm trong khoảng con của ký tự nào $\rightarrow$ Xuất ký tự đó.
 4. Thu hẹp khoảng hiện tại thành đúng khoảng con vừa tìm được.
 5. Lặp lại cho đến khi giải mã đủ số lượng ký tự.

THỰC HÀNH MÃ HÓA SỐ HỌC.

```
def arithmetic_encode(message, probs):  
    low, high = 0.0, 1.0  
    for char in message:  
        range_size = high - low  
        cum_prob = 0  
        for c, p in probs.items():  
            if c == char:  
                high = low + range_size * (cum_prob + p)  
                low = low + range_size * cum_prob  
                break  
            cum_prob += p  
    return (low + high) / 2  
  
probs = {'A': 0.5, 'B': 0.3, 'C': 0.2}  
message = "AB"  
encoded_value = arithmetic_encode(message, probs)  
print(f"Giá trị mã hóa số học của '{message}': {encoded_value}")
```

MÃ LZW (LEMPERL-ZIV-WELCH)

- **Nguyên lý:** Thay thế các chuỗi lặp lại bằng một mã số ngắn hơn.
- **Từ điển động:** LZW xây dựng từ điển trong lúc nén. Khi gặp chuỗi đã xuất hiện, thay bằng mã cố định (thường 9-12 bit).
- **Cách hoạt động:**
 1. Khởi tạo từ điển ban đầu với các ký tự đơn.
 2. Đọc dữ liệu đầu vào để tạo chuỗi hiện tại.
 3. Nếu chuỗi mới đã có trong từ điển, tiếp tục mở rộng.
 4. Nếu chưa có, xuất mã của chuỗi cũ và thêm chuỗi mới vào từ điển.
- **Điểm hay:** Cả bộ nén và giải nén đều tự xây dựng lại cùng một từ điển mà không cần gửi kèm.

VÍ DỤ MÃ HOÁ VỚI LZW

- Mã hoá chuỗi: "ABAABABA"
- Từ điển ban đầu: A: 65, B: 66 (Mã ASCII)
- Các mã mới bắt đầu từ 256.
- Quá trình:
 - Đọc 'A': có trong từ điển.
 - Đọc 'AB': chưa có → Xuất mã của 'A' (65), thêm 'AB' vào từ điển (mã 256).
 - Đọc 'B': có.
 - Đọc 'BA': chưa có → Xuất mã 'B' (66), thêm 'BA' (257).
 - Đọc 'AAB': chưa có → Xuất mã 'A' (65), thêm 'AA' (258).
 - ... Tiếp tục cho đến hết chuỗi.

Các bước mã hóa LZW

Bước	w hiện tại	Ký tự kế tiếp (c)	w+c có trong từ điển?	Xuất mã	Thêm vào từ điển
Khởi tạo	A	B	AB chưa có	65	256 → AB
2	B	A	BA chưa có	66	257 → BA
3	A	A	AA chưa có	65	258 → AA
4	A	B	AB có (256)	-	-
5	AB	A	ABA chưa có	256	259 → ABA
6	A	B	AB có (256)	-	-
7	AB	A	ABA có (259)	-	-
Kết thúc	ABA	Hết chuỗi	-	259	-

GIẢI MÃ LZW

- **Nguyên lý:** Bộ giải nén tự động xây dựng lại từ điển y hệt bộ nén.
- **Quy trình:**
 1. Khởi tạo từ điển với các ký tự đơn.
 2. Đọc mã đầu tiên, xuất chuỗi tương ứng, lưu làm `Chuỗi_trước` .
 3. Đọc mã tiếp theo:
 - *TH1:* Mã đã có $\rightarrow$ Xuất chuỗi. Thêm vào từ điển `Chuỗi_trước` + Ký tự đầu của chuỗi hiện tại.
 - *TH2 (Đặc biệt):* Mã chưa có $\rightarrow$ Thêm `Chuỗi_trước` + Ký tự đầu của `Chuỗi_trước` , sau đó xuất chuỗi này.
 4. Cập nhật `Chuỗi_trước` = chuỗi vừa xuất. Lặp lại.

THỰC HÀNH NÉN LZW.

```
def lzw_compress(text):  
    dict_size = 256  
    dictionary = {chr(i): i for i in range(dict_size)}  
    w = ""  
    compressed = []  
    for c in text:  
        wc = w + c  
        if wc in dictionary:  
            w = wc  
        else:  
            compressed.append(dictionary[w])  
            dictionary[wc] = dict_size  
            dict_size += 1  
            w = c  
    if w:  
        compressed.append(dictionary[w])  
    return compressed  
  
text = "ABAABABA"  
print("Mã LZW:", lzw_compress(text))
```

MÃ HÓA ĐỘ DÀI RUN (RUN-LENGTH ENCODING - RLE)

- **Nguyên lý:** Thay thế chuỗi các pixel giống hệt nhau bằng cặp (Giá trị, Độ dài).
- **Chuẩn CCITT Group 3 & 4:** Dùng cho ảnh nhị phân (Fax).
 - Group 4 sử dụng mã hóa 2D (READ) tham chiếu dòng trước đó để nén tốt hơn.
- **Giải mã:** Đọc từng cặp (Giá trị, Độ dài) và ghi liên tiếp Giá trị đó vào ảnh với số lượng bằng Độ dài.

a	a	a	b	c	c	c	c
---	---	---	---	---	---	---	---



run-length encoding



a	3	b	1	c	4
---	---	---	---	---	---

original data stream: 17 8 54 0 0 0 97 5 16 0 45 23 0 0 0 0 0 3 67 0 0 8 ...

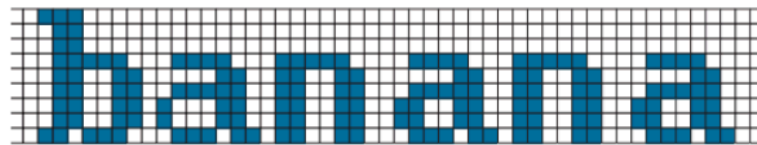
run-length encoded: 17 8 54 0 3 97 5 16 0 1 45 23 0 5 3 67 0 2 8 ...

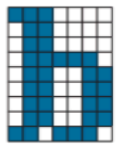
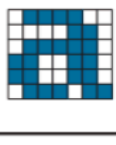
MÃ HÓA SYMBOL-BASED (JBIG2)

- **Nguyên lý:** Tạo từ điển các ký hiệu (ví dụ: ký tự 'a', 'b', hoặc các mẫu lặp lại).
- **Cách hoạt động:** Thay thế các vùng giống nhau trong ảnh bằng tọa độ và token tham chiếu đến từ điển.
- **Ví dụ:** Dữ liệu đầu vào là ảnh quét của chuỗi "banana".

a b c

(a) A bi-level document, (b) symbol dictionary, and (c) the triplets used to locate the symbols in the document.

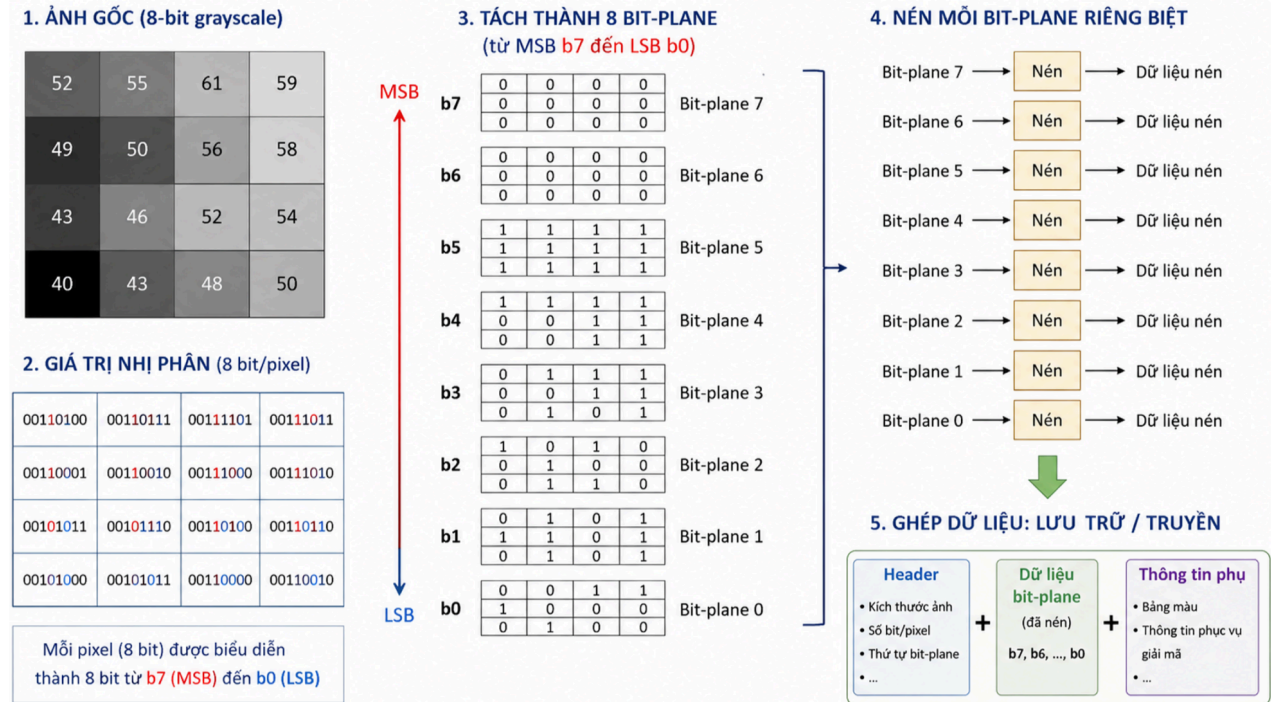


Token	Symbol	Triplet
0		(0, 2, 0) (3, 10, 1) (3, 18, 2) (3, 26, 1) (3, 34, 2) (3, 42, 1)
1		
2		

- **Giải mã:** Giải nén từ điển ký hiệu và luồng tọa độ, sau đó "dán" (paste) các mẫu ký hiệu lên đúng vị trí (X, Y) trên nền trang trắng. Rất hiệu quả cho tài liệu văn bản quét.

MÃ HOÁ BIT-PLANE

- **Khái niệm:** Biểu diễn một ảnh m -bit bằng cách tách nó thành m ảnh nhị phân riêng biệt. Mỗi mặt phẳng bit chứa một bit tại cùng vị trí của tất cả pixel.
- **Ví dụ:** Với ảnh 8 bit, bit cao nhất chứa cấu trúc cường độ chính, các bit thấp hơn mô tả chi tiết tinh và dễ bị nhiễu hơn.
- **Xử lý:** Các bit-plane có thể được nén hoặc xử lý riêng tùy theo mức độ quan trọng.
- **Giải mã:** Dịch chuyển (shift) các bit của từng mặt phẳng nhị phân về lại đúng vị trí, thực hiện phép toán OR (|) để ghép m mặt phẳng thành ảnh gốc.





**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

NÉN CÓ TỔN THẤT (LOSSY)

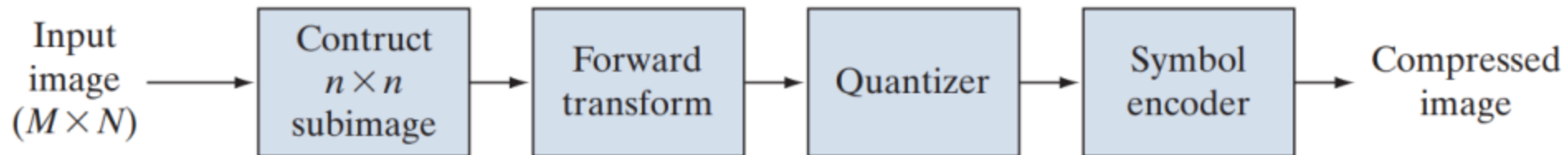


ĐỊNH NGHĨA

- Các phương pháp nén chấp nhận loại bỏ một phần thông tin để đạt tỷ lệ nén cao.
- Tận dụng các đặc điểm của hệ thống thị giác con người (HVS) để loại bỏ thông tin ít nhạy cảm.

MÃ HÓA BIẾN ĐỔI KHỐI (BLOCK TRANSFORM CODING)

- **Mục tiêu:** Loại bỏ dư thừa dữ liệu để giảm dung lượng mà vẫn giữ chất lượng ảnh ở mức chấp nhận được.
- **Quy trình chi tiết:**
 1. **Chia khối (Blocking):** Chia ảnh thành các khối pixel nhỏ (thường 8x8).
 2. **Biến đổi (Transform):** Chuyển đổi giá trị pixel thành hệ số tần số (phổ biến nhất là DCT - Discrete Cosine Transform).
 3. **Lượng tử hóa (Quantization):** Làm tròn các hệ số, loại bỏ thông tin ít quan trọng.
 4. **Mã hóa (Coding):** Mã hóa các hệ số đã lượng tử hóa (thường dùng Zigzag + RLE + Huffman).



- **Phân bổ Bit:** Zonal Coding (giữ hệ số có phương sai lớn nhất) hoặc Threshold Coding (giữ N hệ số có độ lớn tuyệt đối lớn nhất).

- 1. ẢNH GỐC (Ảnh Xăm)**

2. CHIA THÀNH CÁC KHỐI (Block 8x8)

3. LẤY MỘT KHỐI 8x8 (giá trị pixel)

52	55	61	66	70	61	64	73
63	59	55	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	68	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

4. BIẾN ĐỔI DCT 2D (Transform)

415	-30	-61	27	56	-20	-2	0
-22	-22	37	-6	-14	2	1	0
-47	7	77	-24	-29	10	5	0
-49	12	34	-15	-10	6	2	0
12	6	-13	4	7	-2	1	0
-8	-4	-7	2	1	0	0	0
-3	-1	-3	0	0	0	0	0
0	0	0	0	0	0	0	0

(Hệ số tần số: DC ở góc trên trái, AC chiếm phần còn lại)

5. LƯỢNG TỬ HÓA (Quantization)

26	-2	-4	2	3	-1	0	0
-1	-1	2	0	-1	0	0	0
-3	1	5	-1	-2	1	0	0
-3	1	2	-1	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

(Nhiều hệ số tần số cao ≈ 0)

6. QUÉT ZIGZAG (Zigzag Scan)

26	-2	-1	-4	2	-1	0	0	2	1	-3	-1	0	0	0	0	0	0	...	0
----	----	----	----	---	----	---	---	---	---	----	----	---	---	---	---	---	---	-----	---

(Các số 0 liên tiếp ở cuối)

7. MÃ HÓA ENTROPY (RLE + Huffman)

Header (thông tin ảnh, thông số lượng tử, ...)

Bảng Huffman

Dữ liệu entropy (dòng bit nén)

8. LƯU / TRUYỀN

TÓM TẮT QUY TRÌNH

Ảnh gốc → Chia khối → DCT → Lượng tử hóa → Quét zigzag → RLE (zero-run) → Huffman → Dữ liệu nén

Ý tưởng: Tập trung năng lượng vào ít hệ số (tần số thấp), loại bỏ/nén mạnh các hệ số tần số cao.

- **Lượng tử hóa:** Chia các hệ số cho ma trận lượng tử hóa (Q-table) và làm tròn. Nhiều hệ số tần số cao trở thành 0.
- **Mã hóa:** Quét zigzag để gom các số 0 lại, sau đó dùng RLE và Huffman để mã hóa.

GIẢI MÃ BIẾN ĐỔI KHỐI

- Quy trình:

1. **Giải mã Entropy:** Dùng Huffman/Arithmetic để giải mã luồng bit, khôi phục các hệ số DCT.
2. **Giải lượng tử hóa (Inverse Quantization):** Nhân từng hệ số với giá trị tương ứng trong Ma trận lượng tử. (Lưu ý: Bước này gây tổn thất vĩnh viễn, các số 0 không thể khôi phục chính xác).
3. **Biến đổi ngược DCT (IDCT):** Áp dụng IDCT lên khối 8×8 để chuyển ngược về miền không gian.
4. **Ghép khối:** Đặt các khối 8×8 về đúng vị trí để tạo thành bức ảnh hoàn chỉnh.

MÃ HÓA DỰ ĐOÁN (PREDICTIVE CODING/DPCM)

- **Ý tưởng:** Các pixel lân cận thường giống nhau. Thay vì gửi giá trị tuyệt đối, ta chỉ gửi phần chênh lệch (sai số) giữa pixel thật và pixel dự đoán.
- **Nguyên lý:** $e(n) = f(n) - \hat{f}(n)$. Chỉ mã hóa sai số dự đoán $e(n)$.
 - $f(n)$: dữ liệu thực tế.
 - $\hat{f}(n)$: dữ liệu dự đoán.
 - $e(n)$: sai số dự đoán.
- **Phân loại:**
 - *Lossless*: $\hat{f}(n)$ là tổ hợp tuyến tính các pixel lân cận.
 - *Lossy*: Thêm bộ lượng tử hóa cho $e(n)$ để giảm số bit. Bộ lượng tử hóa tối ưu: Lloyd-Max.

VÍ DỤ MÃ HOÁ DỰ ĐOÁN

- Giả sử cần truyền dãy pixel: 150, 152, 149, 151.
- Mã hóa trực tiếp: Mỗi số 8-bit $\rightarrow$ 32 bit.
- Dùng dự đoán đơn giản: $\hat{x}_n = x_{n-1}$
 - Truyền $x_1 = 150$ (8 bit)
 - $e_2 = 152 - 150 = 2$ (chỉ cần 2 bit)
 - $e_3 = 149 - 152 = -3$ (2 bit)
 - $e_4 = 151 - 149 = 2$ (2 bit)
- **Tổng:** $\approx 8 + 2 + 2 + 2 = 14$ bit. Tiết kiệm hơn rất nhiều so với 32 bit.

GIẢI MÃ DỰ ĐOÁN

- **Nguyên lý:** Cộng phần sai số nhận được với giá trị dự đoán.
- **Quy trình:**
 1. Nhận sai số $e(n)$ từ luồng bit.
 2. Tính giá trị dự đoán $\hat{f}(n)$ từ các pixel đã giải nén trước đó.
 3. Khôi phục pixel gốc: $f(n) = \hat{f}(n) + e(n)$.
 4. Lưu $f(n)$ vào bộ nhớ đệm để làm dữ liệu dự đoán cho pixel $n + 1$.

THỰC HÀNH MÃ HÓA VÀ GIẢI MÃ DPCM.

```
def dpcm_encode(signal):
    encoded = [signal[0]]
    for i in range(1, len(signal)):
        predicted = signal[i-1]
        error = signal[i] - predicted
        encoded.append(error)
    return encoded

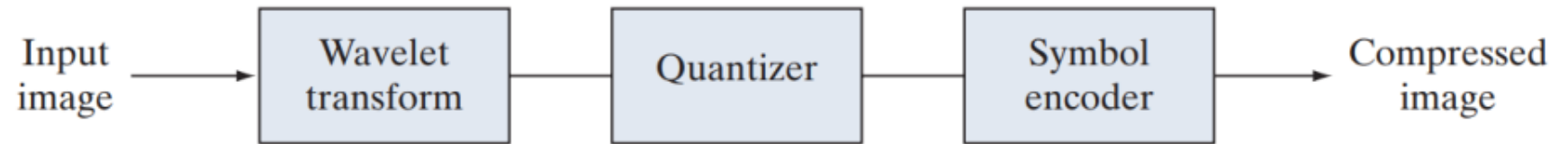
def dpcm_decode(encoded):
    decoded = [encoded[0]]
    for i in range(1, len(encoded)):
        predicted = decoded[i-1]
        decoded.append(predicted + encoded[i])
    return decoded

signal = [150, 152, 149, 151]
encoded = dpcm_encode(signal)
print("Mã hóa DPCM:", encoded)
print("Giải mã DPCM:", dpcm_decode(encoded))
```

MÃ HÓA WAVELET

- **Khái niệm:** Phương pháp nén ảnh dùng biến đổi wavelet rời rạc (DWT). Khắc phục nhược điểm artifact khối của DCT (JPEG) khi nén tỷ lệ cao.
- **Đặc trưng:**
 - Biến đổi toàn ảnh, không chia khối.
 - Biểu diễn đa phân giải (multiresolution).
 - Hỗ trợ cả nén lossless và lossy.
 - Cho phép truyền ảnh tiến triển (progressive).

- **Chuẩn tiêu biểu:**
JPEG 2000.



- **Các bước:**
 1. Biến đổi DWT: Ảnh $\rightarrow$ các băng con hệ số.
 2. Lượng tử hóa: Chia mỗi hệ số cho bước lượng tử.
 3. Mã hóa entropy: Dùng mã hóa bit-plane + mã hóa số học (EZW, SPIHT, EBCOT).

GIẢI MÃ WAVELET

- **Quy trình:**
 1. **Giải mã Entropy:** Giải mã luồng bit (thường dùng Arithmetic Coding kết hợp EZW, SPIHT, EBCOT) để khôi phục các hệ số Wavelet.
 2. **Giải lượng tử hóa:** Nhân các hệ số với bước lượng tử (hoặc dùng bước = 1 nếu là lossless).
 3. **Biến đổi Wavelet ngược (IDWT):** Tổng hợp lại ảnh từ các băng con (sub-bands) tần số thấp và cao để tạo ra ảnh ở độ phân giải gốc.
- **Ưu điểm:** Quá trình này không gây ra hiệu ứng khối (blocking artifact) như JPEG thông thường, cho phép nén tỷ lệ cao mà vẫn giữ được độ mượt của ảnh.